PROJECT DESIGN PHASE

Proposed Solution Template

Date	03 October 2026
Team ID	SWTID-2026-6390
Project Name	Script-Controlled ACL – Restrict Record Access Based on Field Value
Maximum Marks	2 Marks

Proposed Solution

Script-Controlled ACL – Restrict Record Access Based on Field Value

S.No.	Parameter	Description
1	**Problem Statement (Problem to be solved)**	In ServiceNow, a user who can reach the Institution Details table may be able to view or change records that belong to other branches. The table needs a rule that looks at the Branch value, so that EEE users can read only EEE records and the administrator still has full access. In addition, creating, editing and deleting records should be allowed only to users who have been given the matching role.
2	**Idea / Solution description**	The solution uses record-level ACLs on the custom table u_institution_details. First the table is created with a Branch field (ECE, EEE, CSE). Then the roles bb1, bb2, bb3 and bb4 are created. The READ ACL requires bb1, applies the condition Branch = EEE, and runs an advanced script that uses gs.hasRole() to allow the administrator and bb1 users and deny everyone else. Three more ACLs control the other operations: CREATE needs bb2, WRITE needs bb3 and DELETE needs bb4. The administrator keeps unrestricted access. Finally, the setup is tested by impersonating users with different roles and checking the list and form behaviour.
3	**Novelty / Uniqueness**	The project does not rely on a role alone. It joins three ideas: role-based authorization, a field-based record restriction (Branch = EEE), and separate ACLs for each operation. Because of this, giving someone the read role does not give them the right to create, edit or delete, and the READ rule limits which records are visible instead of opening the whole table.
4	**Social Impact / Customer Satisfaction**	Student and institution details stay private because only permitted users can read them. The chance of records being changed or deleted by mistake becomes smaller, since those actions need their own roles. Users get access that matches their responsibility, and the administrator can still manage every record. This builds trust in how the data is handled.
5	**Business Model (Revenue Model)**	This is an academic ServiceNow configuration project, not a finished commercial product, so it does not have its own revenue model. Realistically, the same approach could be used as a security configuration or customization pattern inside a ServiceNow implementation, or as part of security-related implementation services. No financial results are claimed here.
6	**Scalability of the Solution**	The same ACL pattern can be repeated and extended. More branches can be added as new Branch values with their own conditions. More roles can be created for new groups of users. The pattern can be applied to other tables, and extra record-level conditions can be added to the READ ACL when needed. Other CRUD operations can be handled the same way by giving each one its own ACL and role.